



Closures — The Complete Deep Dive

[[Environment]], GC, Memory Leaks, Factories, Currying, Memoization, Debounce/Throttle

Prepared by Claude for Akshay

1. `[[Environment]]` — How Functions "Remember"

Every function has a hidden internal property called `[[Environment]]`. When the engine **creates** a function, it stores a reference to the Lexical Environment that was active at that moment:

```
// When foo is DEFINED (not called), engine does:
// foo.[[Environment]] = currentLexicalEnvironment
const x = 10;
function foo() { console.log(x); }
// foo.[[Environment]] -> Global LexEnv { x: 10 }
// This is a REFERENCE, not a copy!
```

Internal Structure of a Function Object:

```
+-----+
| foo (function object in memory) |
| [[Call]]      -> the code body   |
| [[Environment]] -> LexicalEnv { x: 10 } -|---> parent scope
| name         -> "foo"            |
| length       -> 0                |
+-----+
```

The Closure Mechanism — Step by Step:

```
function outer() {
  const secret = "hidden";
  function inner() { console.log(secret); }
  return inner;
}
const fn = outer();
fn(); // "hidden" <- HOW?
// Step 1: outer() called -> EC with LexEnv { secret: "hidden" }
// Step 2: inner CREATED -> inner.[[Environment]] = outer's LexEnv
// Step 3: inner returned, outer's EC popped off stack
//         BUT outer's LexEnv stays alive (inner references it!)
// Step 4: fn() called -> scope chain: inner's LexEnv
//         -> inner.[[Environment]] (outer's LexEnv) -> found!
```

```
// After outer() returns:
//
// CALL STACK:      HEAP MEMORY:
// +-----+      +-----+
// | Global |      | outer's LexEnv |
// +-----+      | { secret: "hidden" } |
//                +-----^-----+
// fn -> inner function |
//      [[Environment]] -----+
//
// LexEnv alive because [[Environment]] references it!
```

✓ Prove It in Chrome DevTools

Run `console.dir(fn)` and expand the function object. Look for `[[Scopes]]` — that's Chrome's way of showing `[[Environment]]`. You'll see: `Closure (outer) { secret: "hidden" }`

Different Functions = Different `[[Environment]]`:

```
function createAdder(x) {
  return function(y) { return x + y; };
}
```

```
const add5 = createAdder(5); // [[Environment]] -> { x: 5 }  
const add10 = createAdder(10); // [[Environment]] -> { x: 10 }  
add5(3); // 8 (reads x=5 from its own [[Environment]])  
add10(3); // 13 (reads x=10 from its own [[Environment]])
```

2. Closure = Function + Lexical Environment Reference

★ Precise Definition

A **closure** is a function object combined with a reference to the Lexical Environment in which it was created (stored in its `[[Environment]]` internal slot). It's not magic — it's just a function that carries a pointer to its birth environment.

Three Requirements for a Meaningful Closure:

```
function meaningful() {
  const data = "enclosed"; // 1. Outer variable exists
  return function() {      // 2. Inner function references it
    console.log(data);      // 3. Inner function outlives outer
  };
}
const fn = meaningful();    // outer has returned
fn();                       // but data still accessible!
```

Multiple Closures Sharing the Same Environment:

```
function createPair() {
  let shared = 0; // ONE LexicalEnvironment
  return {
    increment: () => ++shared, // all three have
    decrement: () => --shared, // [[Environment]] ->
    getValue: () => shared,    // SAME LexEnv
  };
}
const pair = createPair();
pair.increment(); // 1
pair.increment(); // 2
pair.decrement(); // 1
pair.getValue();  // 1
// All three share SAME 'shared' variable
// Closures capture by REFERENCE, not by value
// If one modifies 'shared', others see the change
```

```
// Memory diagram:
// pair.increment.[[Environment]] -+
// pair.decrement.[[Environment]] -+--> LexEnv { shared: 1 }
// pair.getValue.[[Environment]] -+
```

3. Garbage Collection & Memory Leaks with Closures

JS uses **mark-and-sweep** GC. If nothing references an object, it's freed. Closures prevent GC by holding references via `[[Environment]]`.

```
// Normal: GC frees memory after function returns
function normal() {
  const big = new Array(1000000); // 8MB
  console.log(big.length);
}
normal(); // big has no references -> GC frees it
// Closure: GC CANNOT free memory!
function leaky() {
  const big = new Array(1000000); // 8MB
  return function() { console.log(big.length); };
}
```

```
const fn = leaky();  
// fn.[[Environment]] -> LexEnv -> big (8MB stuck!)  
// To free: fn = null; (breaks the reference chain)
```

Leak Pattern 1: Capturing More Than Needed

```
// BAD: closure captures ENTIRE parent scope  
function process() {  
  const huge = fetchData();          // 100MB  
  const result = transform(huge);     // 1MB  
  return () => result;                // only needs 1MB  
  // But [[Environment]] may keep huge (100MB) alive!  
}  
  
// FIX: isolate what you need in separate function  
function createGetter(result) {  
  return () => result; // [[Environment]] only has result (1MB)  
}  
  
function processFixed() {  
  const huge = fetchData();  
  return createGetter(transform(huge));  
  // huge is NOT in createGetter's scope -> GC frees it!  
}
```

Leak Pattern 2: Forgotten Timers

```
// BAD: interval runs forever -> closure + data live forever
function startMonitoring(data) { // data = 50MB
  setInterval(() => {
    console.log(data.status); // [[Environment]] -> { data: 50MB }
  }, 1000);
  // No way to stop it! Memory leaked.
}
// FIX: return cleanup function
function startFixed(data) {
  const id = setInterval(() => console.log(data.status), 1000);
  return () => clearInterval(id); // cleanup frees closure + data
}
const stop = startFixed(bigData);
stop(); // clears interval -> closure GC'd -> data freed
```

Leak Pattern 3: Event Listeners

```
// BAD: listener keeps componentData alive
function setup(componentData) { // 10MB
  btn.addEventListener("click", () => {
    console.log(componentData.name);
  }); // Even if btn removed from DOM, listener may persist!
}
// FIX: AbortController for clean removal
function setupFixed(componentData) {
  const ac = new AbortController();
  btn.addEventListener("click", () => {
    console.log(componentData.name);
  }, { signal: ac.signal });
  return () => ac.abort(); // removes listener -> GC'd
}
```

Leak Pattern 4: React useEffect

```
// BAD: missing cleanup
useEffect(() => {
  const conn = createConnection(roomId);
  conn.on("message", (msg) => setMessages(m => [...m, msg]));
  conn.connect();
  // NO CLEANUP -> connection + closure live forever!
}, [roomId]);
// FIX: always return cleanup
useEffect(() => {
  const conn = createConnection(roomId);
  conn.on("message", (msg) => setMessages(m => [...m, msg]));
  conn.connect();
  return () => conn.disconnect(); // cleanup!
}, [roomId]);
```

V8's Smart Optimization:

```
function outer() {
  const used = "used";
  const unused = "unused"; // 100MB hypothetically
  return function() {
    debugger;
    console.log(used);
    // V8 sees 'unused' is NOT referenced
    // -> excludes it from [[Environment]]!
```

```
};  
}  
// DevTools Scope: Closure(outer) { used: "used" }  
// 'unused' is NOT there! V8 optimized it away.  
// BUT eval() breaks this optimization:  
// If inner has eval(""), V8 keeps EVERYTHING  
// because it can't know what eval might access.
```

■ Memory Leak Summary

Forgotten timers (setInterval without clearInterval), **Event listeners** not removed, **Large closures** capturing more than needed, **React useEffect** without cleanup function. Always ask: "Will this closure outlive its usefulness? If so, how do I clean it up?"

4. Closure in Loops — The var Trap

Understanding this at the `[[Environment]]` level:

```
// THE BUG: all callbacks see i = 3
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3, 3, 3
// [[Environment]] trace:
// Loop runs in ONE EC. VarEnv: { i: 0 -> 1 -> 2 -> 3 }
// Iter 0: arrow.[[Environment]] = LexEnv -> VarEnv { i }
// Iter 1: arrow.[[Environment]] = SAME LexEnv -> VarEnv { i }
// Iter 2: arrow.[[Environment]] = SAME LexEnv -> VarEnv { i }
// All arrows -> same VarEnv -> i = 3 when they execute
```

```
// FIX 1: let - NEW LexicalEnvironment per iteration
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 0, 1, 2
// Iter 0: NEW LexEnv_0 { i: 0 } -> arrow.[[Env]] = LexEnv_0
// Iter 1: NEW LexEnv_1 { i: 1 } -> arrow.[[Env]] = LexEnv_1
// Iter 2: NEW LexEnv_2 { i: 2 } -> arrow.[[Env]] = LexEnv_2
// Each arrow -> DIFFERENT LexEnv -> its own i
```

```
// FIX 2: IIFE - creates new scope manually
for (var i = 0; i < 3; i++) {
  (function(captured) {
    // NEW EC with LexEnv { captured: 0/1/2 }
    setTimeout(() => console.log(captured), 100);
  })(i); // pass current i by value
}
// FIX 3: Function factory
function createLogger(val) {
  return () => console.log(val);
  // arrow.[[Environment]] -> { val: 0/1/2 }
}
for (var i = 0; i < 3; i++) {
  setTimeout(createLogger(i), 100);
}
```

5. Practical Patterns

5.1 Factory Functions — Private State

```
function createUser(name, role) {
  let loginCount = 0;    // private!
  let lastLogin = null;  // private!
  return {
    getName: () => name,
    login() {
      loginCount++;
      lastLogin = new Date();
      return `${name} logged in (${loginCount}x)`;
    },
    getStats: () => ({ loginCount, lastLogin, name, role }),
  };
}
```



```
};  
}  
const admin = createUser("Akshay", "admin");  
admin.login();           // "Akshay logged in (1x)"  
admin.login();           // "Akshay logged in (2x)"  
admin.loginCount;        // undefined (truly private!)  
admin.getStats();        // { loginCount: 2, ... }
```

```
// Real-world: DB connection factory (Next.js / MERN)  
function createDB(uri) {  
  let conn = null;  
  let queryCount = 0;  
  return {  
    query: async (model, filter) => {  
      if (!conn) conn = await mongoose.connect(uri);  
      queryCount++;  
      return model.find(filter);  
    },  
    getCount: () => queryCount,  
    disconnect: async () => {  
      if (conn) { await mongoose.disconnect(); conn = null; }  
    },  
  };  
}
```

5.2 Partial Application — Pre-fill Arguments

```
function partial(fn, ...presetArgs) {
  return function(...laterArgs) {
    return fn(...presetArgs, ...laterArgs);
  };
}

// API helper: pre-fill method + baseUrl
function request(method, baseUrl, endpoint, data) {
  return fetch(`${baseUrl}${endpoint}`, {
    method,
    body: data ? JSON.stringify(data) : undefined,
  });
}

const apiGet = partial(request, "GET", "https://api.com");
const apiPost = partial(request, "POST", "https://api.com");
await apiGet("/users"); // GET /users
await apiPost("/users", { name: "Akshay" }); // POST

// Logging with context
function log(level, module, msg) {
  console.log(`[${level}] [${module}] ${msg}`);
}

const authLog = partial(log, "INFO", "AUTH");
authLog("User logged in"); // [INFO] [AUTH] User logged in
```

5.3 Currying — f(a, b, c) into f(a)(b)(c)

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args); // all args -> call
    }
    return (...next) => curried(...args, ...next); // partial
  };
}

// Permission checker
function hasPermission(role, resource, action) {
  const perms = {
    admin: { users: ["read", "write", "delete"] },
    viewer: { users: ["read"] },
  };
  return perms[role]?.[resource]?.includes(action) ?? false;
}

const check = curry(hasPermission);
const checkAdmin = check("admin"); // pre-fill role
const adminUsers = check("admin")("users"); // pre-fill role+resource
checkAdmin("users")("delete"); // true
adminUsers("write"); // true
check("viewer", "users", "write"); // false (all at once works too)
```

Feature	Partial Application	Currying
Transforms	$f(a,b,c) \rightarrow f(b,c)$	$f(a,b,c) \rightarrow f(a)(b)(c)$
Returns	Fn taking REST of args	Chain of single-arg fns
Pre-fills	Some args (flexible)	One arg at a time
Use case	Config preset helpers	Composable pipelines

5.4 Memoization — Cache Results

```
function memoize(fn) {
  const cache = new Map(); // closed over, persists!
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key); // HIT!
    const result = fn(...args);
    cache.set(key, result);
    return result; // MISS
  };
}

const factorial = memoize(function(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
});

factorial(5); // calculates 120
factorial(5); // cached! instant
factorial(6); // only calculates 6*factorial(5) (5 is cached)
```

```
// Memoize with TTL (cache expiration)
function memoizeWithTTL(fn, ttlMs = 60000) {
  const cache = new Map();
  return function(...args) {
    const key = JSON.stringify(args);
    const cached = cache.get(key);
    if (cached && Date.now() - cached.ts < ttlMs)
      return cached.value; // still fresh
    const result = fn(...args);
    cache.set(key, { value: result, ts: Date.now() });
    return result;
  };
}

// Cache API results for 5 min
const fetchPrice = memoizeWithTTL(
  async (sym) => (await fetch(`/api/${sym}`)).json(),
  5 * 60 * 1000
);
```

■ React Memoization = Closures

`useMemo(() => expensive(deps), [deps])` — memoize a **value**

`useCallback((args) => fn(args), [deps])` — memoize a **function reference**

Both use closures internally to store the cached result between renders.

5.5 Debounce — Wait Until User Stops

```
function debounce(fn, delay) {
  let timeoutId = null; // closed over, persists!
  function debounced(...args) {
    clearTimeout(timeoutId); // cancel previous
    timeoutId = setTimeout(() => {
      fn.apply(this, args); // fire after silence
      timeoutId = null;
    }, delay);
  }
  debounced.cancel = () => { clearTimeout(timeoutId); };
  debounced.flush = (...args) => {
    clearTimeout(timeoutId);
  };
}
```

```
    fn.apply(this, args); // fire immediately
  };
  return debounced;
}
// Search input: only fires 300ms after last keystroke
const search = debounce(async (q) => {
  displayResults(await (await fetch(`/search?q=${q}`)).json());
}, 300);
input.addEventListener("input", (e) => search(e.target.value));
```

5.6 Throttle — Execute at Fixed Intervals

```
function throttle(fn, limit) {
  let waiting = false; // closed over
  let lastArgs = null; // closed over, stores latest call
  return function(...args) {
    if (waiting) {
      lastArgs = args; // save for trailing call
      return;
    }
    fn.apply(this, args); // execute immediately
    waiting = true;
    setTimeout(() => {
      waiting = false;
      if (lastArgs) {
        fn.apply(this, lastArgs); // trailing call
        lastArgs = null;
      }
    }, limit);
  };
}
// Scroll: fires at most every 100ms
const onScroll = throttle(() => {
  updateNavbar(window.scrollTo);
}, 100);
window.addEventListener("scroll", onScroll);
```

Debounce vs Throttle Visual:

```
// User types: --a--b--c--d-----e--f--
//
// Debounce (300ms):
// --a--b--c--d-----e--f--
//           ^           ^
//           fires d     fires f
// Waits for silence, fires ONCE
//
// Throttle (300ms):
// --a--b--c--d-----e--f--
//   ^       ^       ^   ^   ^
//   fires   fires fires fires fires
// Fires at REGULAR intervals
```

Use Case	Debounce	Throttle
Search input	Yes (wait for typing to stop)	No
Form validation	Yes (validate after pause)	No
Window resize	Yes (recalculate once)	Yes (periodic)
Scroll position	No	Yes (smooth updates)
Button (prevent double)	Yes	Yes
Game input	No	Yes (fixed frame rate)
API rate limiting	No	Yes (max N calls/sec)

6. Complete Closure Mental Model

```
// The Lifecycle of a Closure:
//
// 1. Function CREATED
//   Engine: fn.[[Environment]] = currentLexicalEnvironment
//
// 2. Function RETURNED / stored
//   Parent EC popped off stack
//   Parent's LexEnv stays in HEAP ([[Environment]] refs it)
//
// 3. Function CALLED
//   New EC created
//   Scope chain: own LexEnv -> [[Environment]] -> outer...
//
// 4. Function GARBAGE COLLECTED
//   [[Environment]] reference broken
//   Parent's LexEnv can finally be GC'd
//   All enclosed variables freed
```

All Patterns at a Glance:

Pattern	Closure Stores	Real-World Use
Factory	Private state (count, conn)	createUser, createDB, modules
Partial Application	Pre-filled arguments	API helpers, loggers, configs
Currying	One arg at a time	Permission checks, pipelines
Memoization	Cache (Map)	Expensive calcs, API caching
Debounce	timeoutId	Search input, resize, validation
Throttle	waiting flag + lastArgs	Scroll, mousemove, game loops

7. Cheat Sheet

✓ Quick Rules

1. Every function has `[[Environment]]` — a hidden pointer to its birth scope
2. Closure = function + `[[Environment]]` reference to parent's LexEnv
3. Closures capture by **reference**, not value — shared variables change for all
4. Closures prevent GC — parent's LexEnv stays alive while closure exists
5. `var` in loops = one shared variable. `let` = new LexEnv per iteration
6. Always clean up: `clearInterval`, `removeEventListener`, `useEffect` return
7. V8 optimizes — unreferenced vars excluded, but `eval()` breaks this
8. `console.dir(fn) + expand [[Scopes]]` to SEE closures in DevTools

★ Interview One-Liner

"A closure is a function that retains access to its lexical scope via the `[[Environment]]` internal slot, even after the outer function has returned. This enables patterns like private state (factories), pre-configured functions (partial application), and cached computations (memoization). The tradeoff is memory — closures prevent garbage collection of enclosed variables until the closure itself is dereferenced."